

CẨM NANG ÔN THI VẤN ĐÁP & HƯỚNG DẪN HIỂU CODE

Dự án: Phần mềm Học từ vựng (LearnFlash)

Nền tảng: Kotlin, Android Studio, Jetpack Compose, Room Database, Firebase Firestore, Retrofit API (Coroutines & Flow)

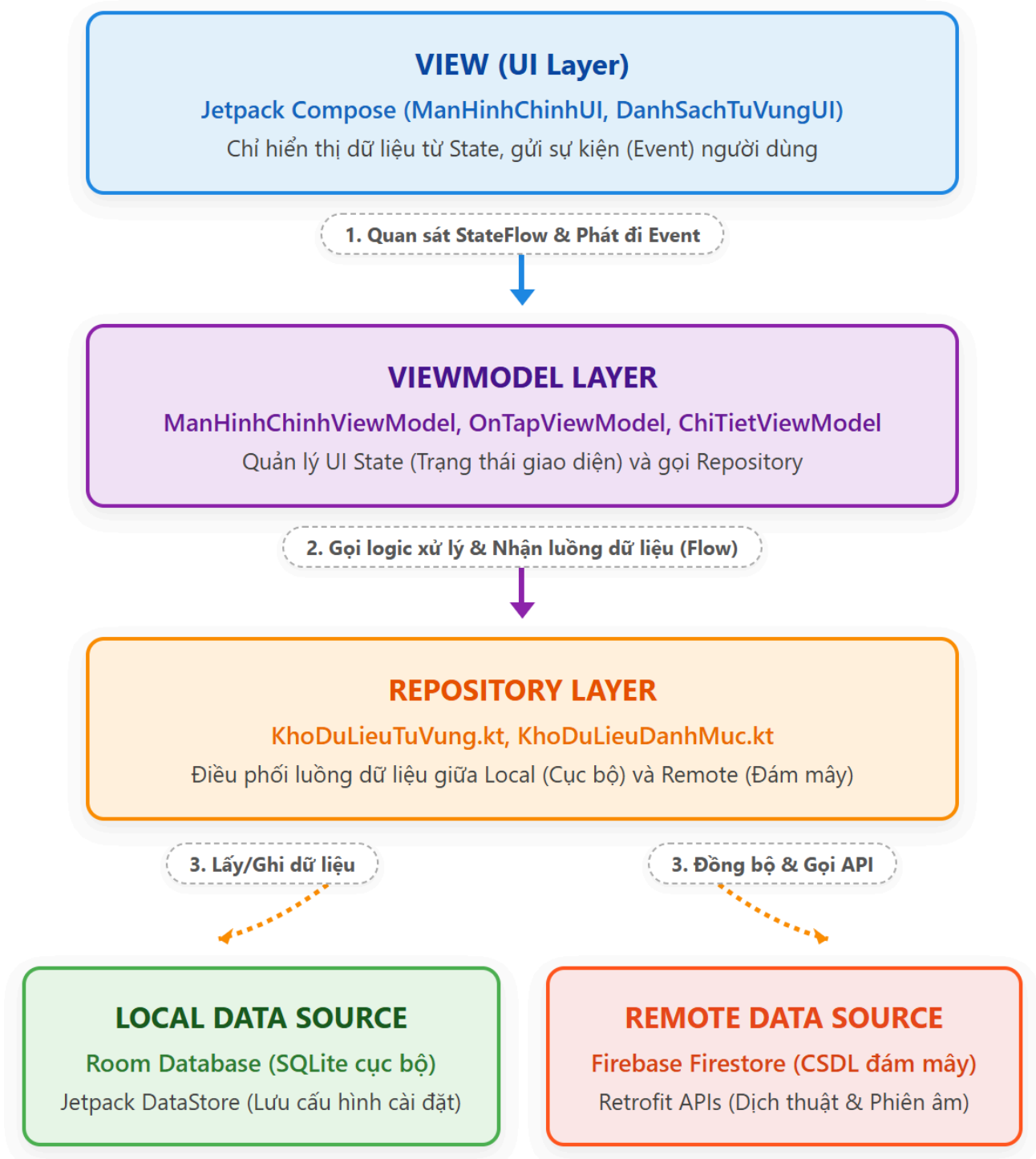
HƯỚNG DẪN ÔN THI THEO PHÂN CÔNG THÀNH VIÊN

Quy trình vấn đáp đã được tách chi tiết thành 3 tài liệu hướng dẫn riêng cho từng thành viên theo đúng phân công công việc:

- Thành viên 1 (UI/UX & Theme):** Xem hướng dẫn ôn tập chi tiết tại file **huong_dan_thanh_vien_1_ui_theme.md**
 - Thành viên 2 (Cấu trúc dữ liệu & APIs):** Xem hướng dẫn ôn tập chi tiết tại file **huong_dan_thanh_vien_2_du_lieu_api.md**
 - Thành viên 3 (Kiểm thử, Validation & Vòng đời):** Xem hướng dẫn ôn tập chi tiết tại file **huong_dan_thanh_vien_3_kiem_thu_vong_doi.md**
-

PHẦN 1: BẢN ĐỒ KIẾN TRÚC DỰ ÁN (MVVM & REPOSITORY PATTERN)

Dự án này áp dụng mô hình kiến trúc **MVVM (Model - View - ViewModel)** kết hợp với **Repository Pattern** chuẩn khuyến nghị của Google để tách biệt rõ ràng luồng dữ liệu và giao diện.



1. Vai trò của từng lớp

- **View (UI Layer):** Xây dựng bằng **Jetpack Compose**. Lớp này hoàn toàn "thụ động" (declarative). Nhiệm vụ của nó là quan sát trạng thái (State) từ ViewModel và vẽ giao diện. Khi người dùng thao tác (nhấn nút, nhập chữ), View sẽ gọi các hàm xử lý sự kiện trong ViewModel. **Tuyệt đối không viết logic xử lý dữ liệu hoặc truy vấn Database trực tiếp ở đây.**
- **ViewModel:** Đóng vai trò là "cầu nối" trung gian. Nó lưu giữ trạng thái giao diện dưới dạng StateFlow hoặc State để tránh mất dữ liệu khi xoay màn hình (Configuration Changes). ViewModel gọi các hàm của Repository và sử dụng viewModelScope để chạy các tác vụ bất đồng bộ (Coroutine).

- **Repository (Kho dữ liệu):** Lớp điều phối dữ liệu. Nó đóng vai trò quyết định dữ liệu nào được lấy từ cơ sở dữ liệu nội bộ (Room DB) để tối ưu tốc độ ngoại tuyến, và khi nào cần đẩy/đồng bộ dữ liệu lên đám mây (Firestore) hoặc gọi các API dịch thuật bên ngoài.
 - **Data Sources (Nguồn dữ liệu):**
 - **Room Database:** Hệ quản trị cơ sở dữ liệu SQLite cục bộ trên máy, đảm bảo ứng dụng chạy mượt mà ngay cả khi không có kết nối Internet (Offline-First).
 - **Firebase Firestore:** Cơ sở dữ liệu đám mây NoSQL dùng để tải danh mục và từ vựng mẫu khi khởi động ứng dụng lần đầu, đồng thời đồng bộ các từ vựng và danh mục mới do người dùng tạo (tiến trình học SRS được cấu hình chỉ lưu cục bộ ở Room DB cục bộ để tránh xung đột dữ liệu).
 - **Retrofit APIs:** Tra cứu phiên âm từ *Free Dictionary API* và dịch nghĩa từ *MyMemory Translation API*.
 - **Jetpack DataStore:** Lưu cấu hình cài đặt gọn nhẹ (ví dụ: trạng thái bật/tắt Dark Mode, số lượng mục tiêu học hàng ngày).
-

PHẦN 2: THUẬT NGỮ KỸ THUẬT CỐT LÕI (ĐỊNH NGHĨA CHÍNH XÁC)

Dưới đây là các thuật ngữ Android, Jetpack Compose và Database cốt lõi được sử dụng trong dự án:

Thuật ngữ	Khái niệm kỹ thuật	Vai trò thực tế trong dự án
Jetpack Compose	Bộ công cụ hiện đại của Google giúp xây dựng giao diện Android theo phong cách khai báo (Declarative UI) bằng code Kotlin hoàn toàn.	Thiết kế toàn bộ màn hình giao diện ứng dụng mà không cần dùng file XML layout truyền thống.
Composable Function	Các hàm được đánh dấu bằng <code>@Composable</code> , đại diện cho một thành phần giao diện có thể tái sử dụng.	Định nghĩa các thành phần UI như <code>ManHinhChinhUI</code> , <code>OnTapUI</code> , các button, hộp thoại cảnh báo lỗi,...
State & Recomposition	State là trạng thái dữ liệu của UI. Recomposition là cơ chế tự động vẽ lại (re-render) các hàm Composable khi dữ liệu State thay đổi.	Khi người dùng nhấn nút lật thẻ flashcard, trạng thái <code> dangLatThe</code> đổi từ <code>false</code> sang <code>true</code> , Compose tự động vẽ lại mặt sau thẻ.
Room Database	Thư viện trừu tượng hóa SQLite (ORM) do Google cung cấp, giúp thao tác với SQLite dễ dàng và an toàn hơn bằng các đối tượng Kotlin.	Quản lý việc lưu trữ từ vựng, danh mục, lịch sử học tập cục bộ dưới máy người dùng.
Entity	Lớp dữ liệu Kotlin được chú thích bằng <code>@Entity</code> , đại diện cho một bảng (table) trong Room Database.	Các lớp như <code>TuVung</code> , <code>DanhMuc</code> đại diện cho bảng tương ứng trong CSDL SQL cục bộ.
DAO (Data Access Object)	Giao diện chứa các câu lệnh truy vấn SQL (Query, Insert, Update, Delete) để tương tác trực tiếp với Database.	Định nghĩa cách lấy danh sách từ cần ôn tập, đếm số từ đã thuộc trong ngày tại <code>TuVungDao</code> .
Firestore	Cơ sở dữ liệu NoSQL dạng tài liệu (Document-based Cloud Database) của Google Firebase.	Lưu trữ từ vựng, danh mục mẫu và đồng bộ danh mục/từ vựng mới lên đám mây (không kèm theo tiến độ học SRS).
Retrofit	Thư viện HTTP Client mạnh mẽ dành cho Android để kết nối và gọi dữ liệu từ RESTful Web APIs.	Thực thi các hàm gọi API bên ngoài để tự động dịch nghĩa tiếng Anh -> tiếng Việt và lấy phiên âm từ vựng.
Coroutine & Suspend	Công cụ xử lý đa luồng (Concurrency) siêu nhẹ của Kotlin.	Dùng để chạy các tác vụ tốn thời gian như ghi Database, tải dữ liệu mạng dưới

Thuật ngữ	Khái niệm kỹ thuật	Vai trò thực tế trong dự án
function	Suspend function là các hàm có thể tạm dừng và tiếp tục chạy mà không làm nghẽn luồng chính.	luồng nền (IO thread), giữ giao diện mượt mà không bị đơ giật.
Flow & StateFlow	Luồng truyền tải dữ liệu bất đồng bộ hoạt động theo cơ chế phản ứng (Reactive stream). StateFlow là một luồng luôn giữ lại giá trị trạng thái mới nhất.	Phát ra danh sách từ vựng từ Room Database đến UI, tự động cập nhật danh sách hiển thị trên DataGridView/LazyColumn bất cứ khi nào CSDL có thay đổi.
Jetpack DataStore	Công cụ lưu trữ dữ liệu dạng Key-Value thế hệ mới, thay thế hoàn toàn SharedPreferences cũ, hỗ trợ hoàn toàn Coroutines và Flow.	Lưu trữ cấu hình giao diện tối (Dark Mode) và số lượng mục tiêu từ vựng học hàng ngày trong CaiDatDataStore .

PHẦN 3: GIẢI THÍCH CÁC KHỐI CODE LẠ & CƠ CHẾ KỸ THUẬT PHỨC TẠP

1. Thuật toán Lặp ngắt quãng (Spaced Repetition System - SRS)

- **Vị trí:** Nằm trong [OnTapViewModel.kt:L83-102](#).
- **Cơ chế hoạt động:**
 Khi học viên lật thẻ ôn tập và chọn đánh giá:
 - Nếu chọn "**Nhớ**" (`nhoDung = true`): Cấp độ SRS (`capDoSrs`) sẽ tăng thêm 1 đơn vị. Thời điểm ôn tập tiếp theo (`ngayOnTapTiepTheo`) sẽ được dời đi xa hơn. Công thức tính: Ngày hiện tại + (Cấp độ SRS mới * 1 ngày) . Khi cấp độ SRS đạt từ 5 trở lên, từ vựng được đánh dấu là "**Đã thuộc hoàn toàn**" (`daThuoc = true`) và sẽ không xuất hiện trong các phiên ôn tập tới hạn nữa.
 - Nếu chọn "**Quên**" (`nhoDung = false`): Cấp độ SRS lập tức **reset về 0**. Thời điểm ôn tập tiếp theo trở về thời gian hiện tại (`System.currentTimeMillis()`), nghĩa là từ đó phải được học lại ngay lập tức.
- **Khối code thực tế:**

```

val capDoMoi = if (nhoDung) tuHienTai.capDoSrs + 1 else 0
val ngayTiepTheo = System.currentTimeMillis() + (capDoMoi * 24 * 60 * 60 * 1000L) ,
val daThuocHoanToan = capDoMoi >= 5

val tuCapNhat = tuHienTai.copy(
    capDoSrs = capDoMoi,
    ngayOnTapTiepTheo = ngayTiepTheo,
    daThuoc = daThuocHoanToan
)
khoDuLieu.capNhatTienDoSrs(tuCapNhat) // Ghi xuống Room Database & đồng bộ lên Fire

```

2. Kỹ thuật Gọi API song song (Parallel Network Calls)

- **Vị trí:** Nằm trong [KhoDuLieuTuVung.kt:L68-99](#).
- **Công dụng:** Khi thêm một từ vựng mới, ứng dụng cần gọi 2 API khác nhau: một API để lấy nghĩa tiếng Việt (MyMemory) và một API để lấy phiên âm/loại từ (Free Dictionary API).
 - Nếu gọi tuần tự (gọi API 1 xong mới gọi API 2), tổng thời gian chờ sẽ bằng Thời gian API 1 + Thời gian API 2 (khoảng 3-4 giây).
 - Bằng cách dùng Coroutine `async` và `await`, cả 2 API được gọi chạy song song cùng một thời điểm. Tổng thời gian chờ chỉ bằng thời gian của API chạy lâu nhất (khoảng 1.5 giây).
- **Khối code thực tế:**

```

coroutineScope {
    // Kích hoạt chạy song song 2 Luồng API
    val congViecDichThuat = async { dichThuatApi.dichVanBan(vanBanNgon = tuKhoa) }
    val congViecPhienAm = async { tuVungApi.traCuuTuVung(tuKhoa) }

    // Đợi cả 2 công việc hoàn thành và lấy kết quả
    val phanHoiDich = congViecDichThuat.await()
    val phanHoiPhienAm = congViecPhienAm.await()
    // ... (xử lý trích xuất dữ liệu JSON đổ vào thực thể TuVung)
}

```

3. Cơ chế Đồng bộ hóa dữ liệu ngoại tuyến (Offline-First)

- **Vị trí:** Nằm trong [FirebaseNgonDuLieu.kt](#) và [KhoDuLieuTuVung.kt](#).
- **Cơ chế hoạt động:**
 - **SQLite/Room làm nguồn chân lý chính (Cache-first):** Ứng dụng luôn đọc dữ liệu từ Room Database cục bộ để hiển thị lên màn hình. Điều này giúp ứng dụng tải cực nhanh và hoạt động bình thường kể cả khi hoàn toàn mất mạng.

- **Bật bộ nhớ đệm Firestore:** Firestore được cấu hình thông qua `persistentCacheSettings {}` để tự động lưu trữ và quản lý cache cục bộ. Khi người dùng thêm từ mới lúc ngoại tuyến, Firestore SDK tự lưu vào hàng đợi cache offline. Ngay khi có mạng lại, hệ thống sẽ tự động đồng bộ phần dữ liệu mới này lên đám mây.
- **Tách biệt tiến trình ôn tập SRS:** Để tránh việc xung đột dữ liệu tiến độ học (do lệch thời gian học hoặc nhiều máy học chung một tài khoản), các trường SRS (`capDoSrs` , `ngayOnTapTiepTheo` , `daThuoc`) chỉ được cập nhật cục bộ dưới Room Database của từng máy. Dòng lệnh đồng bộ SRS lên đám mây đã được tắt: `// firebaseNguonDuLieu.dongBoTienDoSrs(tuVung)` .
- **Cơ chế chống trùng dữ liệu thông minh:** Khi máy đang online và người dùng thêm từ vựng/danh mục mới, trước khi đồng bộ lên Firestore, hệ thống thực hiện 3 bước truy vấn đối soát chống trùng trên server (khớp chính xác, dạng viết thường, dạng viết hoa chữ đầu). Nếu từ vựng/danh mục đó đã tồn tại trên Firestore rồi, nó sẽ tự động bỏ qua không upload lên nữa để tránh trùng lặp dữ liệu đám mây.

PHẦN 4: BỘ CÂU HỎI VẤN ĐÁP THƯỜNG GẶP (Q&A CHI TIẾT)

I. Nhóm Câu hỏi về UI/UX và Jetpack Compose

? Câu 1 (Dễ - Giao diện): Sự khác biệt cơ bản giữa việc xây dựng UI bằng Composable Function so với View truyền thống (XML) là gì?

- **Trả lời:**
 - **XML truyền thống (Imperative UI - Hướng mệnh lệnh):** Giao diện được định nghĩa tĩnh trong các tệp XML. Ở file Java/Kotlin, lập trình viên phải gọi `findViewById()` để liên kết code với UI, sau đó dùng các hàm thay đổi trạng thái thủ công như `setText()` , `setVisibility()` , `setEnabled()` . Điều này rất dễ gây ra lỗi không đồng bộ giữa dữ liệu lưu trong bộ nhớ và giao diện hiển thị thực tế.
 - **Jetpack Compose (Declarative UI - Hướng khai báo):** Giao diện được viết trực tiếp bằng mã nguồn Kotlin thông qua các hàm `@Composable` . Ta không trực tiếp tác động thay đổi các thành phần giao diện mà chỉ khai báo cách UI hiển thị dựa trên dữ liệu đầu vào (State). Khi State thay đổi, Compose sẽ tự động kích hoạt quá trình Recomposition để vẽ lại các Composable bị ảnh hưởng.

? Câu 2 (Dễ - Material 3): Bạn đã áp dụng Material Design 3 như thế nào (ví dụ: sử dụng MaterialTheme để thống nhất màu, font, shape)?

- **Trả lời:**

Hệ thống theme của dự án được cấu hình đồng bộ trong thư mục `tieuChuanGiaoDien`:

 - **MauSac.kt:** Định nghĩa bảng màu Material 3 bao gồm Primary, Secondary, Background,... cho cả 2 chế độ Sáng (`lightColorScheme`) và Tối (`darkColorScheme`).
 - **KieuChu.kt:** Định nghĩa các kiểu chữ tiêu chuẩn (Typography) như TitleLarge, BodyMedium... sử dụng hệ font thống nhất.

- **GiaoDienUngDung.kt**: Cung cấp hàm Composable `GiaoDienLearnFlash` bọc toàn bộ ứng dụng bằng `MaterialTheme(...)`. Nhờ đó, mọi thành phần UI trong ứng dụng (Card, Button, Text,...) đều tự động kế thừa và tuân thủ các quy chuẩn màu sắc, phông chữ và bo góc (shape) của Material 3.

? Câu 3 (Trung bình - Danh sách): Giải thích vai trò của LazyColumn (hoặc LazyGrid) và tại sao nó hiệu quả hơn Column khi ứng dụng của bạn hiển thị danh sách dữ liệu lớn?

- **Trả lời:**
 - **Column**: Sẽ tiến hành khởi tạo và vẽ (compose) toàn bộ các phần tử trong danh sách cùng một lúc vào bộ nhớ RAM, ngay cả những phần tử nằm ở dưới chưa cuộn tới. Nếu danh sách có hàng nghìn từ vựng, ứng dụng sẽ bị tràn bộ nhớ (Out Of Memory) hoặc gây hiện tượng đơ lag nghiêm trọng.
 - **LazyColumn / LazyGrid**: Chỉ tải và vẽ những phần tử hiện đang hiển thị trực tiếp trong khung nhìn màn hình của thiết bị. Khi người dùng cuộn danh sách, các phần tử trôi ra ngoài sẽ được giải phóng vùng nhớ và tái sử dụng (recycle) để nạp nội dung cho các phần tử chuẩn bị xuất hiện. Điều này giúp tối ưu hóa hiệu năng, tiết kiệm bộ nhớ RAM và làm chuyển động cuộn cực kỳ mượt mà.

? Câu 4 (Trung bình - Điều hướng): Làm thế nào bạn quản lý việc điều hướng giữa các màn hình trong ứng dụng (tối thiểu 3 màn hình) theo chuẩn Navigation Compose? Nút Back được xử lý như thế nào?

- **Trả lời:**
 - **Quản lý điều hướng**: Toàn bộ luồng điều hướng được cấu hình tập trung trong file `DieuHuongCumManhinh.kt`. Sử dụng `rememberNavController()` để tạo đối tượng điều phối màn hình, kết hợp với thành phần `NavHost` chứa các route chuỗi như `"manHinhChinh"`, `"thongKe"`, `"gioiThieu"`, `"danhSachTuVung/{danhMucId}/{tenDanhMuc}"`, ... chuyển trang bằng lệnh `navController.navigate("tên_route")`.
 - **Xử lý nút Back**: Nút Back vật lý hoặc nút Back trên thanh tiêu đề (TopAppBar) được liên kết trực tiếp với ngăn xếp (BackStack) của NavController thông qua hàm `navController.popBackStack()`. Tại các màn hình con, ta truyền callback `quayLai = { navController.popBackStack() }` để khi người dùng nhấn nút quay về, hệ thống tự động loại bỏ màn hình hiện tại ra khỏi stack và phục hồi giao diện màn hình trước đó.

? Câu 5 (Khó - Điều hướng): Trình bày cách bạn triển khai "Destination" và "Route". Nêu lợi ích của việc sử dụng các đối tượng định tuyến an toàn (type-safe navigation) thay vì chỉ dùng chuỗi (string routes).

- **Trả lời:**
 - **Triển khai trong dự án**: Hiện tại dự án đang sử dụng định tuyến dạng chuỗi (String-based route). Ví dụ: `composable("chiTiet/{id}/{danhMucIdMacDinh}")`. Các tham số đi kèm được truyền trực tiếp vào chuỗi định vị dưới dạng path parameters, sau đó dùng `navArgument` để ép kiểu dữ liệu mong muốn (như `IntType`, `StringType`) và trích xuất giá trị tại block composable.

- **Lợi ích của Type-Safe Navigation:** Trong các phiên bản Jetpack Navigation mới nhất, Google cho phép sử dụng các lớp đối tượng Kotlin được đánh dấu `@Serializable` (sử dụng thư viện Kotlin Serialization) làm Destination thay cho chuỗi thuần. Lợi ích chính:
 1. **An toàn kiểu dữ liệu (Compile-time safety):** Ngăn chặn lỗi viết sai chính tả chuỗi route hoặc truyền sai kiểu dữ liệu (ví dụ truyền chuỗi văn bản cho tham số khóa chính ID kiểu Int), lỗi được phát hiện ngay khi biên dịch chứ không đợi đến lúc chạy ứng dụng.
 2. **Code tường minh & tự động hóa:** Không cần viết code định cấu hình tham số `navArgument(...)` hay parse chuỗi thủ công, hệ thống tự động ánh xạ đối tượng truyền đi thành tham số đích.

? Câu 6 (Khó - Compose): Giải thích cơ chế **Recomposition** của Compose và cách nó liên quan đến việc cập nhật UI khi State thay đổi.

- **Trả lời:**
 - **Cơ chế Recomposition:** Là quá trình Jetpack Compose tự động gọi lại các hàm Composable khi các tham số đầu vào hoặc trạng thái (State) bên trong nó thay đổi, nhằm mục đích tính toán lại cây giao diện (UI tree) và cập nhật giao diện hiển thị trên màn hình.
 - **Liên quan đến State:** Compose sử dụng mô hình lắng nghe chủ động. Khi một hàm Composable đọc một thuộc tính của đối tượng State (ví dụ: `viewModel1.tuKhoa.value`), hệ thống Compose Runtime sẽ tự động ghi nhận Composable này đang "đăng ký" theo dõi State đó. Bất cứ khi nào State được gán một giá trị mới, Compose sẽ phát hiện và chỉ kích hoạt chạy lại (Recompose) chính xác các Composable có đọc State đó (được gọi là Smart Recomposition), giúp giảm thiểu tối đa tài nguyên dựng hình, tránh hao pin và giật lag thiết bị.

Nhóm II: Kiến trúc (MVVM) và Quản lý State

? Câu 7 (Dễ - MVVM): Trong mô hình MVVM, bạn đặt logic nghiệp vụ và dữ liệu ở đâu, và vai trò của lớp UI (Composable) là gì?

- **Trả lời:**
 - **Vị trí đặt Logic & Dữ liệu:**
 - **Logic dữ liệu cục bộ/mạng:** Nằm ở lớp **Repository** ([KhoDuLieuTuVung.kt](#)), chịu trách nhiệm CRUD database Room, đồng bộ đám mây Firestore và gọi API.
 - **Logic nghiệp vụ & trạng thái UI:** Nằm ở **ViewModel**, tiếp nhận sự kiện từ UI, gọi Repository xử lý và cập nhật lại trạng thái giao diện.
 - **Vai trò của UI (Composable):** Đóng vai trò là tầng **View**, hoàn toàn thụ động. Nó chỉ có nhiệm vụ quan sát trạng thái (State) từ ViewModel để vẽ giao diện và đẩy ngược lại các sự kiện (Click, Text Change) lên ViewModel xử lý thông qua lambda callback. UI tuyệt đối không gọi trực tiếp Database hay xử lý logic nghiệp vụ.

? Câu 8 (Dễ - State): Giải thích khái niệm State trong Compose và cách UI của bạn tự động cập nhật khi State đó thay đổi.

- **Trả lời:**
 - **Khái niệm State:** State (trạng thái) là bất kỳ dữ liệu nào có khả năng thay đổi theo thời gian và tác động trực tiếp đến giao diện hiển thị của ứng dụng (ví dụ: danh sách từ vựng, biến đếm loading, chuỗi thông báo lỗi nhập liệu).
 - **Cơ chế tự động cập nhật:** Trong Compose, ta khai báo biến trạng thái sử dụng `mutableStateOf` hoặc `StateFlow`. Khi một hàm Composable truy cập vào thuộc tính `.value` của State này để vẽ UI, Compose ghi nhận liên kết subscription. Khi code cập nhật giá trị mới cho State, Compose Runtime nhận tín hiệu thay đổi và lập tức lập lịch chạy lại (Recompose) khối giao diện chứa State đó để đồng bộ giao diện hiển thị theo dữ liệu mới nhất.

? Câu 9 (Trung bình - MVVM): Tại sao dữ liệu phải "chảy một chiều" từ ViewModel xuống UI? Lợi ích chính của việc này là gì trong việc duy trì ứng dụng?

- **Trả lời:**
 - **Luồng dữ liệu một chiều (Unidirectional Data Flow - UDF):** Trong kiến trúc MVVM, trạng thái UI (State) chỉ chảy theo chiều đi xuống (ViewModel -> UI) và các sự kiện người dùng (Events) chỉ chảy theo chiều ngược lên (UI -> ViewModel).
 - **Lợi ích chính:**
 1. **Một nguồn chân lý duy nhất (Single Source of Truth):** Giúp giao diện đồng nhất tuyệt đối, không xảy ra lỗi dữ liệu hiển thị trên màn hình khác biệt với dữ liệu lưu trữ thực tế.
 2. **Dễ gỡ lỗi (Easy Debugging):** Khi UI hiển thị sai, ta biết chắc chắn lỗi xuất phát từ việc tính toán trạng thái trong ViewModel hoặc Repository, chỉ cần lần theo luồng đi xuống để sửa lỗi.
 3. **Dễ viết Unit Test:** Tách biệt UI giúp ta có thể viết kiểm thử tự động cho logic thay đổi trạng thái trong ViewModel mà không cần chạy giả lập thiết bị Android để dựng giao diện.

? Câu 10 (Trung bình - Lifecycle): Trình bày cách bạn đảm bảo dữ liệu (trong ViewModel) không bị mất khi người dùng xoay màn hình (configuration change).

- **Trả lời:**
 - Khi thiết bị xảy ra thay đổi cấu hình (xoay màn hình, đổi ngôn ngữ hệ thống), Android OS sẽ tiêu diệt hoàn toàn Activity hiện tại và khởi tạo lại Activity mới để nạp tài nguyên giao diện tương ứng.
 - Để dữ liệu không bị mất, ta lưu trữ dữ liệu trong **ViewModel**. Vòng đời (Lifecycle) của ViewModel được thiết kế sống sót qua các sự kiện thay đổi cấu hình này. Khi Activity bị hủy, ViewModel vẫn được giữ lại an toàn trong bộ nhớ RAM (`ViewModelStore`).
 - Khi Activity mới khởi tạo lại, hệ thống sẽ gán lại instance ViewModel cũ cho Activity mới. Nhờ đó, các State lưu trữ bên trong ViewModel (như danh sách từ vựng, trạng thái form) được giữ nguyên vẹn giúp UI khôi phục trạng thái tức thì mà không cần truy vấn lại CSDL.

? Câu 11 (Khó - Lifecycle): Khi xoay màn hình từ dọc sang ngang, các callback vòng đời nào bạn đã override (ví dụ: onCreate, onStart, onStop, onDestroy,...) đã được gọi và theo thứ tự nào? (Yêu cầu sinh viên đối chiếu với ảnh Logcat đã báo cáo).

• **Trả lời:**

Khi thực hiện xoay thiết bị, Activity cũ bị hủy hoàn toàn và Activity mới được tái tạo. Dựa trên các callback ghi log tại `MainActivity.kt` với tag lọc logcat `LearnFlash_Lifecycle`, thứ tự cuộc gọi vòng đời diễn ra chính xác như sau:

1. onPause : Activity mất tiêu điểm tương tác.
2. onStop : Giao diện Activity cũ bị ẩn hoàn toàn khỏi màn hình.
3. onDestroy : Hủy hoàn toàn thực thể Activity cũ khỏi bộ nhớ RAM để giải phóng tài nguyên.
4. onCreate : Khởi tạo thực thể Activity mới, thiết lập lại nội dung giao diện bằng Compose.
5. onStart : Giao diện Activity mới hiển thị trở lại với người dùng.
6. onResume : Activity mới sẵn sàng bắt đầu tiếp nhận các sự kiện tương tác chạm.

? Câu 12 (Khó - State): Phân biệt giữa StateFlow và LiveData (hoặc MutableState) trong việc quản lý State trong ViewModel. Tại sao bạn chọn loại State đó cho dự án này?

• **Trả lời:**

◦ **Phân biệt:**

- LiveData : Là thành phần cũ thuộc gói Android Jetpack Lifecycle. Nó gắn chặt với vòng đời Android, chỉ có thể cập nhật dữ liệu trên luồng chính (Main Thread) và thiếu các toán tử xử lý luồng dữ liệu nâng cao (như combine, flatMap).
- StateFlow : Thuộc thư viện Kotlin Coroutines Flow, hoạt động bất kể nền tảng (sử dụng được cho Kotlin Multiplatform), không phụ thuộc vào thư viện Android. Nó luôn yêu cầu một giá trị mặc định lúc khởi tạo và cung cấp bộ toán tử phong phú để biến đổi luồng dữ liệu.
- MutableState : Là kiểu dữ liệu cơ bản của riêng Compose Runtime, tối ưu hóa tối đa cho việc Recompose, sử dụng cú pháp rất ngắn gọn (bằng từ khóa `by remember / mutableStateOf`).

◦ **Lý do chọn lựa:**

- Dự án sử dụng **MutableState** trong các ViewModel (như `ChiTietViewModel` hay `OnTapViewModel`) để quản lý các biến trạng thái giao diện đơn giản (như text nhập vào, trạng thái mở dialog) để code gọn gàng, Compose tự động recompose nhanh nhất.
- Dự án sử dụng **StateFlow** để quản lý các dòng dữ liệu dài hoặc danh sách thực thể lấy ra từ Room Database, vì Room hỗ trợ xuất trực tiếp kiểu `Flow`, giúp tận dụng tối đa cơ chế đa luồng bất đồng bộ của Coroutine trước khi đồng bộ nạp lên UI qua hàm `.collectAsState()`.

Nhóm III: Chức năng, Thống kê và Validation

? Câu 13 (Dễ - CRUD): Kể tên và trình bày ngắn gọn về một chức năng CRUD (Thêm, Sửa, Xóa) mà bạn thấy quan trọng nhất trong ứng dụng của mình.

- **Trả lời:**

Chức năng **Cập nhật thông tin từ vựng (Sửa)** tại màn hình [ChiTietUI.kt](#) là quan trọng nhất:

- Nó cho phép người dùng tùy biến nội dung học tập (sửa nghĩa dịch, sửa phiên âm, thay đổi loại từ hoặc đổi danh mục chủ đề).
- **Điểm nghiệp vụ đặc biệt:** Khi bấm Sửa, ViewModel [ChiTietViewModel.kt](#) sẽ lấy dữ liệu từ vựng cũ lên form. Khi lưu lại, hệ thống giữ nguyên các trường tiến độ học SRS cũ (`capDoSrs` , `ngayOnTapTiepTheo` , `daThuoc`) thay vì ghi đè reset về 0, đảm bảo tiến trình ôn tập Spaced Repetition của người dùng không bị mất đi.

? Câu 14 (Dễ - Xử lý sự kiện): Bạn đã sử dụng cơ chế nào để xử lý các sự kiện người dùng như click, nhập liệu, v.v.?

- **Trả lời:**

Ứng dụng sử dụng cơ chế truyền các **sự kiện callback lambda** từ View lên ViewModel:

- **Nhập liệu:** Ở [ChiTietUI.kt](#) , ô nhập liệu `OutlinedTextField` đăng ký sự kiện thay đổi text `onValueChange = { viewModel.capNhatTuKhoa(it) }` . Giá trị mới được đẩy ngay lên ViewModel để ghi nhận vào State.
- **Click chuột:** Nút bấm dịch nghĩa hay nút lưu đăng ký sự kiện `onClick = { viewModel.traCuuVaDich() }` hoặc `onClick = { viewModel.luuTuVung { quayLai() } }` . ViewModel sẽ tiếp nhận sự kiện để chạy hàm logic xử lý tương ứng.

? Câu 15 (Trung bình - Thống kê): Trình bày cách bạn tính toán một trong các chỉ số thống kê (Tổng số bản ghi, giá trị TB, Min/Max) bằng logic Kotlin.

- **Trả lời:**

Các chỉ số thống kê được tính toán trực tiếp thông qua các câu lệnh truy vấn SQL của Room DB tại [TuVungDao.kt](#):

- **Tổng số từ vựng:** `SELECT COUNT(*) FROM tu_vung`
- **Cấp độ SRS trung bình:** `SELECT AVG(capDoSrs) FROM tu_vung`
- **Cấp độ SRS cao nhất:** `SELECT MAX(capDoSrs) FROM tu_vung`
- **Cấp độ SRS thấp nhất:** `SELECT MIN(capDoSrs) FROM tu_vung`

Tất cả các câu truy vấn này trả về dạng `Flow<T>` , được Repository truyền dẫn và ViewModel thu thập để hiển thị trực tiếp lên giao diện thống kê [ThongKeUI.kt](#) bất cứ khi nào CSDL Room có cập nhật mới.

? Câu 16 (Trung bình - Validation): Giải thích cách bạn thực hiện Validation (kiểm tra hợp lệ) cho một trường dữ liệu cụ thể và cách bạn hiển thị thông báo lỗi (dùng `SnackBar/AlertDialog/TextField error`).

- **Trả lời:**

- **Thực hiện Validation:** Trong `ChiTietViewModel.kt:L139-148`, khi người dùng nhấn nút Lưu, ViewModel sẽ kiểm tra xem từ khóa hoặc ý nghĩa có bị để trống hay không:
 - `if (_tuKhoa.value.isBlank()) { _loiNhapLieu.value = "Từ khóa không được để trống"; return }`
 - `if (_nghiaTiengViet.value.isBlank()) { _loiNhapLieu.value = "Ý nghĩa không được để trống"; return }`
- **Cách hiển thị thông báo lỗi:**
 - Trạng thái lỗi được lưu vào biến State `loiNhapLieu` trong ViewModel.
 - Ở `ChiTietUI.kt`, ta gán thuộc tính `isError = viewModel.loiNhapLieu.value.isNotEmpty()` vào ô nhập `OutlinedTextField` để hệ thống tự động đổi màu đường viền sang màu đỏ khi lỗi xảy ra.
 - Đồng thời hiển thị dòng chữ thông báo lỗi màu đỏ bằng thuộc tính `supportingText = { Text(text = viewModel.loiNhapLieu.value, color = MaterialTheme.colorScheme.error) }` ngay dưới ô nhập liệu tương ứng.

? Câu 17 (Khó - Chức năng phức tạp): Trình bày chi tiết luồng hoạt động của chức năng Xuất/Nhập dữ liệu (CSV hoặc JSON) thông qua Storage Access Framework (SAF). Nếu hoạt động này tốn thời gian, bạn xử lý nó như thế nào để không chặn luồng chính (UI thread)?

• **Trả lời:**

- **Luồng hoạt động Xuất/Nhập thông qua SAF và Dialog nâng cao:**

1. **Luồng Xuất (JSON):**

- Ở `GioiThieuUI.kt`, khi người dùng nhấn nút xuất, ứng dụng sử dụng `rememberLauncherForActivityResult` với contract `ActivityResultContracts.CreateDocument("application/json")` để mở trình chọn tệp hệ thống Android (SAF). Người dùng tự chọn thư mục lưu trữ và đặt tên tệp tin.
- Khi nhận được `Uri` phản hồi, ViewModel sẽ kiểm tra xem cơ sở dữ liệu Room có rỗng hay không. Nếu rỗng, ứng dụng sẽ báo lỗi ngay lập tức mà không ghi file.
- Nếu hợp lệ, hệ thống mở Coroutine thông qua `viewModelScope.launch` để đọc dữ liệu Room, chuyển thành chuỗi JSON, và ghi trực tiếp vào Stream ghi tệp của ContentResolver (`context.contentResolver.openOutputStream(uri)`).

2. **Luồng Nhập dữ liệu nâng cao:**

- Khi người dùng nhấn nút nhập, ứng dụng sử dụng `ActivityResultContracts.GetContent()` để kích hoạt trình duyệt tệp hệ thống cho phép chọn tệp tin JSON bất kỳ.
- Khi có `Uri` của tệp tin, giao diện sẽ hiển thị một `AlertDialog` cho phép người dùng lựa chọn một trong 3 chế độ nạp:
 - **Chỉ thêm từ mới:** Bỏ qua các từ vựng đã trùng từ khóa, chỉ thêm các từ mới chưa có trong máy.
 - **Cập nhật đề từ trùng:** Nếu trùng từ khóa, sẽ cập nhật lại nghĩa tiếng Việt, loại từ, phiên âm nhưng giữ nguyên ID và tiến trình SRS hiện tại trong máy.

- **Thay thế toàn bộ:** Xóa sạch toàn bộ từ vựng hiện có trong cơ sở dữ liệu cục bộ và nạp mới hoàn toàn từ tệp tin.
- Sau khi người dùng xác nhận chế độ nạp, ViewModel đọc tệp tin qua `context.contentResolver.openInputStream(uri)`, phân tích cú pháp JSON và tiến hành nạp vào cơ sở dữ liệu Room theo chế độ tương ứng.
- **Xử lý không chặn luồng chính (UI Thread):** Toàn bộ quá trình đọc/ghi file qua luồng IO (Disk I/O), xử lý chuỗi và tương tác Room Database đều được thực hiện bên trong các Coroutine bất đồng bộ được quản lý bởi `viewModelScope.launch`. Trong suốt quá trình xử lý, một cờ trạng thái `đangXuLy` được gán bằng `true` để hiển thị lớp phủ xoay Loading (`ManHinhChoLoading`) trên UI. Do các tác vụ nặng chạy dưới luồng nền (IO Thread), luồng chính (Main Thread) hoàn toàn không bị chặn, giao diện vẫn phản hồi mượt mà và không gây ra lỗi ANR (Application Not Responding).

Nhóm IV: Yêu cầu Nâng cao (A, B, C)

? Câu 18 (Dễ - Retrofit/Room): Retrofit (hoặc Room Database) được dùng để làm gì trong ứng dụng của bạn?

- **Trả lời:**
 - **Room Database:** Dùng để xây dựng kho lưu trữ dữ liệu cục bộ dưới máy người dùng. Room đóng vai trò chuyển đổi các thực thể Kotlin thành bảng CSDL SQLite vật lý, cho phép thêm, sửa, xóa, tìm kiếm từ vựng và danh mục mà không cần kết nối mạng.
 - **Retrofit:** Dùng để kết nối và gọi dữ liệu từ các dịch vụ API bên ngoài thông qua mạng Internet. Cụ thể: gửi yêu cầu lấy nghĩa tiếng Việt từ dịch vụ dịch thuật *MyMemory API*, và lấy phiên âm/loại từ tiếng Anh từ *Free Dictionary API*.

? Câu 19 (Dễ - DataStore): Bạn đã sử dụng DataStore để lưu trữ những loại dữ liệu gì? (Ví dụ: lưu cài đặt người dùng).

- **Trả lời:**

Trong file `CaiDatDataStore.kt`, DataStore được cấu hình lưu trữ 2 loại dữ liệu cài đặt người dùng gọn nhẹ:

 1. Cài đặt chế độ giao diện tối (Dark Mode): biến kiểu Boolean `GIAO_DIEN_TOI_KEY`.
 2. Mục tiêu số từ vựng cần ôn tập tối thiểu trong một ngày: biến kiểu Int `MUC_TIEU_HANG_NGAY_KEY`.

? Câu 20 (Trung bình - Repository): Giải thích vai trò của lớp Repository. Nó giúp tách biệt nguồn dữ liệu như thế nào trong kiến trúc ứng dụng của bạn?

- **Trả lời:**
 - **Vai trò:** Lớp Repository (`KhoDuLieuTuVung.kt`) đóng vai trò là điểm truy cập dữ liệu duy nhất cho tầng ViewModel. Nó chứa toàn bộ logic điều phối và quyết định nguồn dữ liệu.
 - **Tách biệt nguồn dữ liệu:** ViewModel không cần quan tâm dữ liệu từ vựng được lấy từ Room DB dưới máy, tải từ Firestore trên mây, hay vừa dịch từ mạng API về. ViewModel chỉ việc gọi các

phương thức chung của Repository như `lưuTuVung()` hay `traCuuVaDich()`. Repository bên dưới tự phân phối tác vụ: lưu vào Room, đồng thời đẩy lên Firestore, hay gọi API Retrofit song song. Nhờ vậy, tầng giao diện và tầng nghiệp vụ dữ liệu hoàn toàn độc lập, rất dễ bảo trì hoặc thay đổi cơ chế lưu trữ sau này.

? Câu 21 (Trung bình - Xử lý mạng): Làm thế nào ứng dụng của bạn hoạt động được ngay cả khi không có kết nối mạng? (Hỏi về chiến lược truy cập dữ liệu: **Network first** hay **Cache first**).

• **Trả lời:**

- Ứng dụng áp dụng chiến lược truy xuất **Cache-first (Room Database cục bộ trước)**. Khi người dùng mở màn hình chính hoặc danh sách từ vựng, giao diện luôn truy vấn nạp dữ liệu trực tiếp lấy ra từ SQLite của Room DB cục bộ, giúp ứng dụng hoạt động tức thì, mượt mà mà hoàn toàn không phụ thuộc mạng.
- Đối với cơ sở dữ liệu đám mây Firebase Firestore: SDK được cấu hình kích hoạt tính năng lưu trữ bộ nhớ đệm ngoại tuyến (`isPersistenceEnabled = true` thông qua `persistentCacheSettings`). Do đó, khi mất kết nối mạng, các thao tác tạo danh mục hoặc tạo từ mới vẫn được ghi nhận vào cache Firestore cục bộ bình thường. Khi thiết bị có mạng trở lại, SDK Firestore tự động chạy ngầm để đồng bộ hóa phần thay đổi lệch này lên máy chủ đám mây (lưu ý: tiến trình ôn tập SRS được cấu hình chỉ lưu cục bộ dưới Room Database của từng máy chứ không đồng bộ lên Firestore).

? Câu 22 (Khó - Coroutines): Trình bày cách bạn sử dụng Coroutines để xử lý các thao tác bất đồng bộ (ví dụ: gọi API) và cách bạn quản lý việc hiển thị trạng thái Loading/Error cho người dùng.

• **Trả lời:**

- **Xử lý bất đồng bộ:** Khi cần gọi API tra cứu mạng trong `ChiTietViewModel.kt`, ta mở một Coroutine chạy nền bằng `viewModelScope.launch`. Tác vụ gọi API Retrofit được khai báo dưới dạng hàm `suspend` để nó tự giải phóng Main Thread trong lúc đợi phản hồi từ server.
- **Quản lý hiển thị Trạng thái:**
 1. **Loading:** Khi vừa gọi hàm, gán `_dangTai.value = true` để UI hiển thị vòng xoay tiến trình loading.
 2. **Xử lý và Error:** Đặt khối gọi mạng trong khối `try-catch` hoặc xử lý kết quả bằng đối tượng tiện ích `Result`. Nếu thành công, gán giá trị trả về vào các trường dữ liệu; nếu xảy ra lỗi (mất mạng, quá hạn), bắt exception và gán thông báo lỗi vào `_loiNhapLieu.value` để UI hiển thị dòng text báo lỗi đỏ.
 3. **Tắt Loading:** Trong khối `finally` (hoặc cuối hàm xử lý), gán `_dangTai.value = false` để tắt vòng xoay loading trên UI.

? Câu 23 (Khó - Tính năng bổ sung): Nếu bạn chọn Dark/Light theme, hãy giải thích luồng logic đã triển khai để chuyển đổi theme và cách các Composable tự phản hồi lại sự thay đổi này.

• **Trả lời:**

Luồng logic chuyển đổi theme hoạt động khép kín như sau:

1. Ở giao diện cài đặt (`CaiDatUI`), khi người dùng bấm công tắc bật/tắt chế độ tối, `ViewModel` gọi `CaiDatDataStore` để ghi giá trị boolean (true/false) của cấu hình giao diện tối vào file `Preferences`.
2. Tại `MainActivity.kt`, ứng dụng theo dõi luồng cấu hình này bằng Flow: `val giaoDienToi by caiDatDataStore.giaoDienToiFlow.collectAsState(initial = false)`.
3. Biến trạng thái `giaoDienToi` này được truyền thẳng làm đối số đầu vào cho hàm bọc giao diện `GiaoDienLearnFlash(toanGiaoDienToi = giaoDienToi)`.
4. Bên trong `GiaoDienLearnFlash`, hệ màu của ứng dụng (`ColorScheme`) được chọn động tương ứng giữa `lightColorScheme` (chế độ sáng) và `darkColorScheme` (chế độ tối) để nạp vào `MaterialTheme`.
5. Vì các Composable con trong ứng dụng sử dụng các màu hệ thống động (ví dụ: `color = MaterialTheme.colorScheme.background`), khi giá trị `giaoDienToi` thay đổi, Compose Runtime tự động chạy `Recomposition` vẽ lại toàn bộ các Composable con với bảng màu mới tương ứng.

Nhóm V: Nhóm Câu hỏi về Luồng Xử lý chức năng (Flow Tracing & Debugging)

? Câu 24 (Dễ - State & Event): (Chỉ vào một Composable có sự kiện click đơn giản): Khi người dùng nhấn vào Button này, bạn đã truyền sự kiện (lambda/callback) đến ViewModel như thế nào? State nào trong ViewModel đã thay đổi và UI phản ứng (Recompose) ra sao?

- **Trả lời:**
 - **Ví dụ:** Sự kiện nhấn nút "Tra cứu & Dịch" trong màn hình `ChiTietUI.kt`.
 - **Truyền sự kiện:** Tại Composable nút bấm, ta định nghĩa tham số sự kiện `onClick = { viewModel.traCuuVaDich() }`. Khi người dùng click, sự kiện callback lambda được thực thi và gọi trực tiếp phương thức trong ViewModel.
 - **Thay đổi State:** Trong ViewModel, biến trạng thái xoay loading `_dangTai.value` chuyển thành `true`. Sau khi API phản hồi thành công, các State `_nghiaTiengViet.value`, `_phienAm.value`, `_loaiTu.value` được cập nhật dữ liệu tương ứng.
 - **UI Phản ứng:** Các Composable TextField đang đăng ký đọc các biến State này (như `value = viewModel.nghiaTiengViet.value`) phát hiện dữ liệu thay đổi, kích hoạt `Recompose` vẽ lại các ô nhập liệu chứa chuỗi ký tự vừa tải về, và tắt vòng xoay loading.

? Câu 25 (Trung bình - CRUD Flow): (Chỉ vào nút "Lưu" hoặc "Xóa" một bản ghi): Hãy mô tả luồng xử lý (các hàm được gọi) từ UI → ViewModel → Repository → Database (Room/Retrofit). Mã nguồn (code) của hàm xử lý trong Repository có gì đặc biệt (ví dụ: dùng `suspend fun`)?

- **Trả lời:**
 - **Mô tả luồng xử lý khi nhấn nút "Lưu" từ vừng:**
 1. **UI (`ChiTietUI.kt`):** Người dùng nhấn nút Lưu -> Gọi callback `onClick = { viewModel.luuTuVung { quayLai() } }`.
 2. **ViewModel (`ChiTietViewModel.kt`):** Hàm `luuTuVung` kiểm tra tính hợp lệ dữ liệu, đóng gói dữ liệu thành thực thể `TuVung` rồi khởi chạy Coroutine để gọi


```
khoDuLieu.luuTuVung(tuVungLuu) .
```

3. **Repository (KhoDuLieuTuVung.kt)**: Hàm `luuTuVung` tiếp nhận đối tượng. Nếu `id == 0` (từ mới), Repository gọi `tuVungDao.themTuVung(tuVung)` để ghi vào Room DB, và gọi `firebaseNguonDuLieu.themTuVungLenFirestore(tuVung)` để đồng bộ đám mây.

4. **Database (Room & Firestore)**: Câu lệnh SQL `@Insert` được Room biên dịch chạy ghi xuống file SQLite dưới máy, đồng thời SDK Firestore đẩy bản ghi lên server database online.

- **Điểm đặc biệt ở Repository**: Hàm xử lý trong Repository được khai báo là **suspend fun**. Điểm đặc biệt của nó là hàm có thể tạm hoãn (suspend) luồng xử lý của Coroutine hiện tại để chờ tác vụ ghi ổ đĩa hoặc kết nối mạng hoàn thành, sau đó tự phục hồi chạy tiếp mà hoàn toàn không gây block (khóa cứng) luồng giao diện chính (UI Thread).

? Câu 26 (Trung bình - Trích xuất Log): (Liên quan đến Yêu cầu Vòng đời): Hãy chỉ ra 4 callback vòng đời bạn đã override (ví dụ: `onCreate`, `onPause`, `onStop`, `onDestroy`) trong code. Sau đó, mở Logcat và thực hiện một thao tác (ví dụ: xoay màn hình) để minh họa thứ tự các log đã được ghi lại.

• **Trả lời:**

- **Vị trí trong mã nguồn**: Các callback được override trực tiếp tại lớp `MainActivity.kt` bao gồm: `onCreate()` (dòng 28), `onStart()` (dòng 73), `onResume()` (dòng 79), `onPause()` (dòng 85), `onStop()` (dòng 91), `onDestroy()` (dòng 97).
- **Minh họa Logcat khi xoay màn hình**: Mở công cụ Logcat trong Android Studio, nhập bộ lọc tag: `LearnFlash_Lifecycle`. Tiến hành xoay ngang thiết bị di động, ta sẽ thấy các dòng log được ghi lại theo thứ tự tuyến tính:
 - Trạng thái: `onPause` - Tạm dừng các tác vụ chiếm tài nguyên UI
 - Trạng thái: `onStop` - Bắt đầu giải phóng tài nguyên hiển thị tạm thời
 - Trạng thái: `onDestroy` - Xóa bỏ hoàn toàn Activity khỏi bộ nhớ máy
 - Trạng thái: `onCreate` - Khởi tạo tài nguyên giao diện và các phụ thuộc hệ thống
 - Trạng thái: `onStart` - Giao diện bắt đầu hiển thị với người dùng
 - Trạng thái: `onResume` - Sẵn sàng bắt sự kiện tương tác chạm

? Câu 27 (Khó - Bất đồng bộ & Debug): (Nếu dùng Room/Retrofit): Trong Logcat, bạn kiểm tra thế nào để biết rằng thao tác gọi API hoặc thao tác Room Database đã thành công? Hãy chỉ ra đoạn code Coroutine/Repository nơi bạn quản lý việc chuyển trạng thái từ Loading sang Success (hoặc Error).

• **Trả lời:**

- **Cách kiểm tra trong Logcat**: Mở Logcat, gõ bộ lọc tag: `LearnFlash_Firebase` hoặc tag: `LearnFlash_Lifecycle`. Khi gọi API hoặc Room thành công, các đoạn log debug tại `FirestoreNguonDuLieu.kt` sẽ in ra (ví dụ: `"Đã tải 3000 từ vựng từ Firestore (batch insert)"`) hoặc `"Đồng bộ SRS lên Firestore thành công: hello"`). Nếu lỗi xảy ra, Logcat sẽ in log cảnh báo (`Log.w` hoặc `Log.e`) chứa nội dung lỗi kèm stacktrace.
- **Quản lý chuyển trạng thái trong ViewModel**: Ở `ChiTietViewModel.kt:L122-135`:

```

_dangTai.value = true // ==> CHUYỂN SANG TRẠNG THÁI LOADING (HIỂN THỊ VÒNG XOAY
viewModelScope.launch {
    val ketQua = khoDuLieu.traCuuVaDich(_tuKhoa.value.trim())
    if (ketQua.isSuccess) { // ==> TRẠNG THÁI SUCCESS (ĐÃ LẤY ĐƯỢC DỮ LIỆU TỪ AF
        val tuVungTraVe = ketQua.getOrNull()
        _nghiaTiengViet.value = tuVungTraVe?.nghiaTiengViet ?: ""
        _phienAm.value = tuVungTraVe?.phienAm ?: ""
        _loaiTu.value = tuVungTraVe?.loaiTu ?: ""
    } else { // ==> TRẠNG THÁI ERROR (GẶP LỖI MẤT MẠNG HOẶC API QUÁ HẠN SỬ DỤNG)
        _loiNhapLieu.value = ketQua.exceptionOrNull()?.message ?: "Lỗi kết nối n
    }
    _dangTai.value = false // ==> TẮT TRẠNG THÁI LOADING
}

```

? Câu 28 (Khó - Xử lý ngoại lệ): (Chỉ vào màn hình nhập liệu): Khi người dùng nhập dữ liệu không hợp lệ (Validation lỗi), State nào được cập nhật trong ViewModel để thông báo lỗi? Bạn đã sử dụng thành phần UI nào (SnackBar/AlertDialog/TextField error) để hiển thị thông báo đó và luồng dữ liệu của thông báo này chạy như thế nào?

- **Trả lời:**

- **State cập nhật:** Biến trạng thái `_loiNhapLieu` (kiểu `mutableStateOf("")`) trong `ChiTietViewModel.kt` sẽ được cập nhật nội dung chuỗi thông báo lỗi (ví dụ: "Từ khóa không được để trống").
- **Thành phần UI sử dụng:**
 1. Dùng thuộc tính `isError` của `OutlinedTextField` liên kết với State lỗi: `isError = viewModel.loiNhapLieu.value.isNotEmpty()`. Khi có lỗi, khung viền ô nhập liệu lập tức đổi sang màu đỏ.
 2. Dùng thuộc tính `supportingText` của `OutlinedTextField` để hiển thị trực tiếp chuỗi lỗi màu đỏ ngay sát phía dưới ô nhập liệu: `supportingText = { Text(text = viewModel.loiNhapLieu.value, color = MaterialTheme.colorScheme.error) }`.
- **Luồng chạy của dữ liệu:**
 Người dùng bấm nút Lưu -> Kích hoạt sự kiện gọi hàm `luuTuVung` của ViewModel -> Khối `validate` kiểm tra thấy từ khóa trống -> Cập nhật chuỗi lỗi vào `_loiNhapLieu.value` -> Compose Runtime phát hiện State thay đổi giá trị -> Kích hoạt `Recomposition` vẽ lại màn hình `ChiTietUI.kt` -> Khung viền ô nhập chuyển sang màu đỏ và dòng chữ báo lỗi màu đỏ lập tức hiển thị để cảnh báo người dùng.

VI. Nhóm Câu hỏi về Kịch bản thực tế & Đồng bộ hóa dữ liệu (Scenario-Based)

? Câu 29 (Tình huống đồng bộ 2 máy): Giả sử người dùng tải app lần đầu, đã tải xong dữ liệu mặc định (trên cả 2 máy). Khi cả 2 máy đều đang **BẬT** mạng:

1. Nếu máy 1 xóa đi 10 từ vựng trong một danh mục nào đó, máy 2 có bị xóa theo không?
2. Nếu máy 1 thêm 10 từ vựng mới vào một danh mục nào đó, máy 2 có tự động có 10 từ đó không? Sau đó, nếu máy 3 mới tải app lần đầu (bật mạng) thì máy 3 này sẽ tải về bao nhiêu từ vựng (ví dụ ban đầu là 1700 từ)?
3. Nếu máy 1 thêm 1 danh mục mới, máy 2 có danh mục đó không?

• **Trả lời:**

1. **Máy 2 KHÔNG bị xóa 10 từ đó:**

- *Lý do:* Phương thức `xoaTuVung()` trong `KhoDuLieuTuVung.kt` chỉ gọi lệnh xóa cục bộ dưới Room Database của máy 1 (`tuVungDao.xoaTuVung(tuVung)`), hoàn toàn không gửi yêu cầu xóa lên server Firestore.
- Đồng thời, máy 2 không thiết lập trình lắng nghe thời gian thực (Realtime Listener - `addSnapshotListener`) để theo dõi các thay đổi từ Firestore, nên máy 2 sẽ giữ nguyên dữ liệu cục bộ của mình.

2. **Máy 2 KHÔNG tự động có 10 từ mới đó. Nhưng máy 3 tải lần đầu SẼ tải được đầy đủ 1710 từ:**

- *Đối với máy 2:* Máy 2 đã có sẵn dữ liệu trong Room cục bộ, do đó khi khởi động, hàm `khoiTaoDuLieuMacDinh()` sẽ kiểm tra thấy số từ > 0 và tự động bỏ qua việc tải dữ liệu từ Firestore. Máy 2 không có Listener cập nhật thời gian thực nên sẽ không nhận được 10 từ mới.
- *Đối với máy 3:* Do máy 3 tải app lần đầu, Room DB trống rỗng (`demTongSoTuVungMacDinh() == 0`). Hệ thống sẽ chạy hàm `taiTuVungTuFirestore()` thực hiện lấy toàn bộ các document hiện có trong collection `tuVung` trên Firestore bằng lệnh `db.collection(COLLECTION_TU_VUNG).get().await()` . Vì máy 1 đã đẩy 10 từ mới lên Firestore thành công trước đó (nâng tổng số từ trên Firestore từ 1700 lên 1710), máy 3 sẽ tải về đầy đủ **1710 từ vựng**.

3. **Máy 2 KHÔNG tự động cập nhật ngay lập tức danh mục đó. Tuy nhiên, danh mục mới ĐÃ được đồng bộ ngầm lên Firestore Server:**

- *Lý do:* Khi máy 1 thêm danh mục mới, Repository `lưuDanhMuc()` sẽ tự động gọi `themDanhMucLenFirestore()` để đẩy danh mục lên Firestore (sau khi đối soát chống trùng tên).
- Máy 2 đã chạy và khởi tạo dữ liệu Room từ trước nên sẽ không tự động kéo lại dữ liệu từ Firestore. Nhưng nếu máy 3 tải app lần đầu (hoặc nếu ta cài lại app), máy 3 sẽ tải về được cả danh mục mới này từ Firestore.

? Câu 30 (Tình huống xuất/nhập file offline): Máy 2 tải app trước, bật mạng, có khoảng 2000 từ vựng mặc định, chọn xuất file JSON rồi gửi file đó cho máy 1. Máy 1 tải app nhưng TẮT mạng (nên Room trống, không có dữ liệu tải từ Firestore).

1. Nếu máy 1 thực hiện nhập (import) file JSON nhận được từ máy 2 khi vẫn TẮT mạng, máy 1 có khoảng 2000 từ vựng đó không?
2. Sau khi nhập file thành công, máy 1 mới BẬT mạng lên thì điều gì sẽ xảy ra? Dữ liệu 2000 từ tải lên từ máy 1 có bị trùng lặp với các từ đã có sẵn trên Firestore Server không?
3. Nếu sau đó máy 3 mới tải app lần đầu thì máy 3 sẽ nạp bao nhiêu từ từ Firestore? Có bị trùng lặp trong Room của máy 3 không?

• Trả lời:

1. **ĐÚNG**, máy 1 sẽ có khoảng 2000 từ vựng đó trong Room:

- Lý do: Nghiệp vụ nhập dữ liệu trong `GioiThieuViewModel.kt` thực hiện đọc văn bản từ file, chuyển đổi ngược (deserialize) thành danh sách đối tượng `TuVung` và gọi `khoDuLieu.luuTuVung()` với ID gán bằng 0 để ghi trực tiếp xuống SQLite cục bộ thông qua Room. Thao tác ghi SQLite này hoạt động offline 100% nên dữ liệu sẽ được lưu trữ thành công vào máy 1.

2. Khi máy 1 BẬT mạng lên, dữ liệu SẼ KHÔNG bị trùng lặp trên Firestore Server (nhờ cơ chế đối soát chống trùng trực tuyến mới):

- Không bị tải trùng dưới máy 1: Hàm kiểm tra `tuVungDao.demTongSoTuVungMacDinh()` trả về 2000 (lớn hơn 0). Hệ thống xác định máy đã có dữ liệu và tự động bỏ qua bước tải từ Firestore về máy 1.
- Không bị trùng lặp trên Firestore: Trong lúc offline, hàm `luuTuVung()` đã gọi `firebaseNguonDuLieu.themTuVungLenFirestore()` 2000 lần và được Firestore SDK lưu tạm các tác vụ này vào cache offline. Ngay khi máy 1 bật mạng lên, Firestore SDK tự động chạy ngầm các tác vụ này. Tuy nhiên, nhờ cơ chế lọc chống trùng online mới trong `themTuVungLenFirestore()`, trước khi tải lên, hệ thống sẽ thực hiện truy vấn so khớp (3 bước: khớp chính xác, viết thường, viết hoa chữ đầu). Nếu từ vựng đó đã tồn tại trên Firestore rồi (do máy 2 tải lên trước đó), hệ thống sẽ bỏ qua và không thực hiện tải lên nữa. Do đó, **loại bỏ hoàn toàn được hiện tượng trùng lặp dữ liệu trên Firestore Server**.

3. Máy 3 tải app sau đó sẽ nạp chính xác dữ liệu gốc (2000 từ vựng) và KHÔNG bị trùng lặp trong Room:

- Số lượng tải về: Nhờ cơ chế chống trùng lặp online của máy 1, trên Firestore Server chỉ tồn tại duy nhất 1 bản ghi cho mỗi từ vựng gốc (không bị nhân đôi thành 4000 từ). Do đó, máy 3 cài app lần đầu chỉ tải về chính xác **2000 từ vựng**.
- Không bị trùng lặp: Room DB của máy 3 lưu trữ chính xác 2000 từ vựng này không có bất kỳ bản ghi trùng lặp nào.

? Câu 31 (Tình huống đồng bộ tiến độ học SRS): Giả sử lúc đầu hệ thống thiết lập tải tiến độ học tập (SRS) lên Firestore. Hãy phân tích các xung đột xảy ra và giải pháp khắc phục đã triển khai trong mã nguồn dự án.

1. Nếu đồng bộ SRS lên Firestore, xung đột gì sẽ xảy ra khi nhiều máy cùng học hoặc học lệch thời gian?
2. Cách giải quyết triệt để vấn đề này trong mã nguồn dự án là gì?

• **Trả lời:**

1. Các xung đột xảy ra khi đồng bộ SRS lên mây:

- **Xung đột học lệch:** Máy 1 học từ "apple" lên cấp 3 và lưu lên Firestore. Máy 2 tải app lần đầu sẽ tải về cấp độ 3 của máy 1. Nếu sau đó máy 2 ôn tập từ này lên cấp 4, máy 2 sẽ ghi đè lên Firestore. Máy 1 đã có dữ liệu trong Room nên sẽ không bao giờ kéo lại dữ liệu từ Firestore, dẫn đến dữ liệu tiến độ của máy 1 (cấp 3) và máy 2 (cấp 4) bị lệch nhau hoàn toàn.
- **Xung đột ghi đồng thời (Write Conflict):** Nếu cả 2 máy cùng online và cùng ôn tập từ "apple" tại một thời điểm, cả 2 sẽ gửi lệnh ghi đè `update()` lên Firestore. Máy nào gửi lệnh chậm hơn vài phần trăm giây sẽ ghi đè và xóa mất tiến trình học của máy gửi trước (Last Write Wins).

2. Giải pháp khắc phục triệt để đã triển khai:

- **Giữ tiến độ SRS hoạt động 100% cục bộ (Local-only):** Trong lớp Repository [KhoDuLieuTuVung.kt](#), tại hàm `capNhatTienDoSrs()`, ta đã tắt (comment out) dòng gọi lệnh đồng bộ lên Firestore: `// firebaseNgonDuLieu.dongBoTienDoSrs(tuVung)`. Tiến trình học từ vựng lúc này chỉ lưu trên Room Database cục bộ của từng thiết bị.
- **Không đẩy trường SRS khi thêm từ mới:** Tại lớp [FirebaseNgonDuLieu.kt](#), hàm `themTuVungLenFirestore()` đã loại bỏ các trường `capDoSrs`, `ngayOnTapTiepTheo`, `daThuoc` khỏi map dữ liệu gửi đi.
- **Cơ chế Default Giá trị:** Khi thiết bị khác tải từ vựng mới từ Firestore về máy, do các tài liệu trên Firestore không còn chứa các trường tiến độ SRS, hàm `taiTuVungTuFirestore()` sẽ sử dụng các toán tử null-coalescing (`?:`) để tự động gán giá trị mặc định lúc ban đầu cho từ vựng cục bộ (`capDoSrs = 0`, `daThuoc = false` và `ngayOnTapTiepTheo = System.currentTimeMillis()`).
- **Ý nghĩa:** Giải pháp này giúp đảm bảo **tiến trình học tập mang tính cá nhân hóa 100% trên từng thiết bị của mỗi người dùng**, loại bỏ hoàn toàn các xung đột ghi đè hoặc nạp nhầm tiến trình học của người khác, trong khi các từ vựng mới và danh mục chủ đề vẫn được đồng bộ chia sẻ chung bình thường.

? Câu 32 (Tình huống máy ảo tải app lần đầu sau khi đổi code): Máy chính của tôi hiện tại đang có dữ liệu tiến độ ôn tập (đã được đồng bộ lên Firestore từ trước khi đổi code). Nếu một thiết bị khác (hoặc máy ảo) chạy code mới này và tải app lần đầu (bật mạng), thiết bị đó có bị ảnh hưởng bởi tiến độ cũ của tôi trên Firestore không?

• **Trả lời:**

KHÔNG BỊ ẢNH HƯỞNG.

- *Lý do:* Mặc dù các bản ghi từ vựng cũ đã lưu trên Firestore Server trước đây vẫn còn chứa 3 trường tiến độ SRS (`capDoSrs` , `ngayOnTapTiepTheo` , `daThuoc`) chứa dữ liệu cũ của bạn.
- Tuy nhiên, trong mã nguồn mới của hàm `taiTuVungTuFirestore()` , chúng ta đã sửa đổi để **ép trực tiếp** giá trị các trường này về mặc định cục bộ khi tải xuống:
 - `capDoSrs = 0`
 - `ngayOnTapTiepTheo = System.currentTimeMillis()`
 - `daThuoc = false`
- Do đó, khi thiết bị khác hoặc máy ảo tải app và nạp dữ liệu lần đầu, chương trình sẽ bỏ qua hoàn toàn các giá trị SRS cũ lưu trên Firestore và khởi tạo tiến trình học hoàn toàn mới tinh (0 tiến độ) cho máy đó.

? Câu 33 (Tình huống xử lý dữ liệu nhập tệp tin không hợp lệ): Giả sử người dùng nhập tệp tin JSON hoặc CSV bị khuyết các trường bắt buộc (như từ khóa, nghĩa dịch rỗng), hoặc chứa mã danh mục (`danhMucId`) không tồn tại trong máy (hoặc bị rỗng), hệ thống sẽ xử lý thế nào để tránh gây lỗi ứng dụng?

• **Trả lời:**

Ứng dụng đã được tích hợp bộ lọc chuẩn hóa dữ liệu đầu vào (`chuanHoaVaKiemTraDanhSach`) để tự động xử lý các tình huống dữ liệu lỗi một cách thông minh:

1. **Nếu khuyết trường bắt buộc (`tuKhoa` hoặc `nghiaTiengViet` bị rỗng):**

- *Cách xử lý:* Hệ thống sẽ bỏ qua bản ghi từ vựng này (sử dụng `mapNotNull` và trả về `null`). Việc này giúp loại bỏ hoàn toàn các từ vựng rác làm hỏng trải nghiệm học.

2. **Nếu trường danh mục bị rỗng hoặc chứa `danhMucId` chưa tồn tại trong máy:**

- *Nếu rỗng:* Hệ thống tự động đưa từ vựng đó vào danh mục mặc định là `"mac_dinh"` (danh mục "Chung").
- *Nếu chứa `danhMucId` không rỗng nhưng chưa tồn tại trong máy:* Hệ thống sẽ tự động tạo mới một danh mục mới trong cơ sở dữ liệu Room (lưu cục bộ và đồng bộ lên Firestore ngầm) với ID chính là `danhMucId` đó và tên hiển thị tương ứng (viết hoa chữ cái đầu), rồi liên kết từ vựng đó tới danh mục mới này. Cách xử lý này vừa giúp bảo toàn nguyên vẹn cấu trúc phân loại danh mục từ file nhập, vừa tránh được lỗi "từ vựng mồ côi" (không thể hiển thị do trỏ tới danh mục không tồn tại).